p0690r0
Tearable Atomics

Published Proposal, 18 June 2017

This version:
http://wg21.link/P0690

Authors:
JF Bastien (Apple)
Billy Robert O'Neal III (Microsoft)

Audience:
SG1

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Source:
github.com/jfbastien/papers/blob/master/source/P0690r0.bs

Abstract

Atomics which can tear—which are more relaxed than relaxed—seem useless. This paper shows otherwise.

Table of Contents

§ 1. Background

Is it useful for C++ to support "tearable" atomic memory ordering, where the access participates in atomic ordering as strongly as `memory_order_relaxed` accesses, but where the memory is allowed to tear (i.e. isn't single-copy atomic). In C++ standards speak: particular atomic object are **not** indivisible with respect to all other atomic accesses to that object.

Indeed, advanced concurrency and parallelism users will sometimes find a need for objects which are accessed by multiple threads, yet either:

1. Rely on separate atomic objects to provide inter-thread observability guarantees; or
2. Use lock-free accesses on a memory locations on which they would also like to speculate.

What we describe often amounts to a *data race*:

- Two accesses to the same *memory location* by different threads are *not ordered*
- At least one of them stores to the memory location
- At least one of them is not a synchronization action

Data races are *undefined behavior*, and it is often said that there is no such thing as a "benign" data race [benign]. Issues that arise when mixing atomic and non-atomic accesses have been discussed before in the context of the C language's memory model [n4136].

Reconciling these types of issue is discussed in the concurrency and parallelism group from time to time, and so far only one-off solutions have been proposed to the Committee, or the problem has been punted. We believe that this proposal can fix this interesting problem once and for all by looking at how other areas have solved this issue.

After all, this has been solved in non-C++ contexts. To assembly programmers, or to those used to memory models such as Linux's memory model [p0124r2], the distinctions the Standard makes seems overly complex. Their code simply defines atomicity as a property or *code* rather than C++'s definition of atomicity as a property of *particular memory locations* during an object's lifetime. Indeed, in assembly a memory location can be concurrently accessed with a regular non-atomic memory instruction as well as an atomic memory instruction.

§ 2. Usecases

Sample usecases include:

1. Speculative load before a compare-and-exchange instruction
2. Sequence locks
3. Work-stealing deque

Others exist, but we will focus on these three.

§ 2.1. Speculative Load

Consider the following code:

```
struct alignas(sizeof(intptr_t) * 2) Node {
    intptr_t l, r;
};

Node action(const Node& old);

void do_action(std::atomic<Node> *n) {
    Node old = n->load(std::memory_order_relaxed); // Relaxed loads can't t
    while (!n->compare_exchange_weak(old, action(old), std::memory_order_re
                                     std::memory_order_acquire))
      ;
}
};
```

In this example, all lock-free operations (including load / store) *must* be implemented as a compare-and-exchange or load-linked / store-conditional:

- On recent x86-64 using `cmpxchg16b`.
- On A32 without LPAE using `ldrexd`, `clrex`, and `strexd`.

The relaxed memory access could instead speculate by using a tearable load / store, potentially cheaper than a compare-and-exchange or load-link / store-conditional, as long as a compare-and-exchange retry loop follows it to handle races. If tearing occurs then the compare-and-exchange does the right thing.

- On x86-64 using two `movq` instructions (two instructions are never locked and can tear).
- On A32 using `ldrd` (without LPAE the instruction isn't single-copy atomic).

§ 2.2. Seqlock

In the case of sequence locks, the data being protected can be accessed non-atomically and is known to be race-free if the sequence number hasn't changed before and after the data was retrieved, and if it isn't "tagged" as being modified (below, by being odd):

```
template<typename T>
struct Data {
  std::atomic<unsigned> sequence_number = 0;
  std::atomic<T> value0;
  std::atomic<T> value1;
};

std::tuple<T, T> reader(const Data& data) {
  T value0, value1;
  unsigned sequence_before, sequence_after;
  do {
    sequence_before = data.sequence_number.load(std::memory_order_acquire);
    value0 = data.value0.load(std::memory_order_relaxed);
    value1 = data.value1.load(std::memory_order_relaxed);
    std::atomic_thread_fence(std::memory_order_acquire);
    sequence_after = data.sequence_number.load(std::memory_order_relaxed);
  } while (sequence_before != sequence_after || sequence_before & 1);
  return {value0, value1};
}

void writer(Data& data, T value0, T value1) {
  auto sequence_start = data.sequence_number.load(std::memory_order_relaxed
  data.sequence_number.store(sequence_start + 1, std::memory_order_relaxed)
  data.value0.store(value0, std::memory_order_release);
  data.value1.store(value1, std::memory_order_release);
  data.sequence_number.store(sequence_start + 2, std::memory_order_release)
}
```

Notice that in C++ the values being protected must be atomic because this algorithm doesn't use more common acquire / release patterns which C++ encourages. Doing otherwise would be a data race

according to the memory model. One would need to add fences for non-atomic accesses to not be racy.

A more in-depth discussion of seqlock [seqlock] is available.

For the purpose of our discussion, it is especially interesting to considers value types `T` which are never lock-free.

§ 2.3. Work-Stealing Deque

It appears intended that implementations of the parallelism TS [N4578] back scheduling of partitioned work using an ABP work-stealing scheduler, originally described in [ThreadSched].

Under this model, each thread has a local deque of work where it can access the "bottom" of the deque without any synchronization overhead, but other threads can concurrently remove work from the "top". A similar data structure was presented implementable on hardware without double-wide CAS instructions in [WSDeque].

In the Chase-Lev deque, the "top" counter is used to track concurrent access to the top of the deque; and access to the actual elements in the top of the deque is unsynchronized. From the original paper:

```
public Object steal() {
    long t = this.top;                  // 11.
    long b = this.bottom;               // 12.
    CircularArray a = this.activeArray; // 13.
    long size = b - t;                  // 14.
    if (size <= 0) return Empty;        // 15.
    Object o = a.get(t);                // 16.
    if (! casTop(t, t+1))               // 17.
        return Abort;                   // 18.
    return o;                           // 19.
}
```

or translated to C++:

```
variant<empty_t, abort_t, T> steal() {
    int64_t t = top.load();                    // 11.
    int64_t b = bottom.load();                 // 12.
    T * a = activeArray.load();                // 13.
    int64_t size = b - t;                      // 14.
    if (size <= 0) return empty_t{};           // 15.
    T o = a[t % aSize];                        // 16.
    if (! top.compare_exchange_strong(t, t+1)) // 17.
        return abort_t{};                      // 18.
    return o;                                  // 19.
}
```

If the deque contains only one element, this causes undefined behavior in C++'s memory model, because the element accessed on line 16 may be concurrently written by the owning thread of the deque. However, if a data race occurs, the CAS on line 17 fails, and the result of this speculative read is never observed. Only one thread can win the CAS race to increment top.

Of note, imposing that `T` be an atomic type in this instance defeats the entire purpose of using the work-stealing deque, as it would require the owning "bottom" thread to use synchronized access to read from the bottom (including potentially taking a lock of `T` is large; egregious for a data structure whose purpose is to be lock-free) in the uncontended cases, even though the correctness of the algorithm is maintained by discarding any potentially torn results.

§ 3. Further Considerations

Extrapolating from the above examples, it is also useful to consider a few extra usecases where:

- Alignment of the datastructures is purposefully *not* natural. In contrast, `std::atomic` is specified as always being suitably aligned by the implementation.
- Padding of the datastructure isn't the same as that mandated by `std::atomic` (although padding bits are their own bag of special [p0528r0]).
- The datastructure isn't always accessed by memory operations of the same byte-size. This could occur without dangerous type aliasing by using properly type-punned `union`, `memcpy`, or `std::variant`, as well as with SIMD types that sometimes perform element accesses.
- The datastructure being accessed is large, making it non-lock-free and requiring an implementation-provided lock. Many implementations rely on lock sharding for this, but some embed a lock in every large `std::atomic` object.

§ 4. Solutions

There are many solutions to this problem. This paper hopes to round up what has been suggested before, leading to a discussion in the concurrency and parallelism group. This discussion should end in straw polls which provide guidance on where the committee would like to go next with this issue.

1. Atomic views [p0019r5] tackle some of the issues discussed here, but in an environment where data access patterns follow *epochs*. For parts of runtime the view are accessed non-atomically, and for other parts of runtime they are accessed atomically. Atomic views do not solve the general problem discussed here.
2. A paper on thin air values [n3710] discussed adding `non_atomic_load()`, `non_atomic_store()`, and `race_or<T>` type (similar to `std::optional` or `std::expected` but for racy / indeterminate results).
3. Safe `memcpy` [p0603r0] proposes addressing the seqlock example with `nonatomic_load()` and `nonatomic_store()` functions.
4. We also offer a different approach: a new memory order type, `memory_order_tearing`, which has the same semantics as `memory_order_relaxed` but which is allowed to tear. And—of course—`memory_order_tearing` has the neat properly of being spelled with the same number of characters as the other 6 memory orderings.

Not all of these approaches address all the issues discussed above—e.g. `memory_order_tearing` does not address the issue of large non-lock-free `T`—we therefore hope that the concurency and parallelism group will find the wisdom required to weigh each issue and decide which solution fits them best.

§ References

§ Informative References

[BENIGN]
Hans-J. Boehm. How to miscompile programs with “benign” data races. May 2011. URL: http://hboehm.info/boehm-hotpar11.pdf

[N3710]
H. Boehm, et al.. Specifying the absence of "out of thin air" results (LWG2265). URL: https://wg21.link/n3710

[N4136]
M. Batty, P. Sewell, et al.. C Concurrency Challenges Draft. 13 October 2014. URL: https://wg21.link/n4136

[N4578]

Jared Hoberock. Working Draft, Technical Specification for C++ Extensions for Parallelism Version 2. 22 February 2016. URL: https://wg21.link/n4578

[P0019R5]

H. Carter Edwards, Hans Boehm, Olivier Giroux, James Reus. Atomic View. URL: https://wg21.link/p0019r5

[P0124R2]

Paul E. McKenney, Ulrich Weigand, Andrea Parri, Boqun Feng. Linux-Kernel Memory Model. 26 June 2016. URL: https://wg21.link/p0124r2

[P0528R0]

JF Bastien, Michael Spencer. The Curious Case of Padding Bits, Featuring Atomic Compare-and-Exchange. 12 November 2016. URL: https://wg21.link/p0528r0

[P0603R0]

Andrew Hunter. safe memcpy: A simpler implementation primitive for seqlock and friends. URL: https://wg21.link/p0603r0

[SEQLOCK]

Hans-J. Boehm. Can Seqlocks Get Along with Programming Language Memory Models?. 16 June 2012. URL: http://safari.ece.cmu.edu/MSPC2012/slides_posters/boehm-slides.pdf

[ThreadSched]

Nimar S. Arora; Robert D. Blumofe; C. Greg Plaxton. Thread Scheduling for Multiprogrammed Microprocessors. June 1998. URL: https://www.eecis.udel.edu/~cavazos/cisc879-spring2008/papers/arora98thread.pdf

[WSDeque]

David Chase; Yossi Lev. Dynamic Circular Work-Stealing Deque. July 2005. URL: http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.170.1097&rep=rep1&type=pdf